C PROGRAM MINI PROJECT

PASSWORD STRENGTH CHECKER AND GENERATOR

PROJECT TITLE: PASSWORD STRENGTH CHECKER AND GENERATOR

STUDENT NAME: GENGADHARAN S

REGISTER NUMBER: 921724102048

DEPARTMENT:CSE

COLLEGE NAME: SETHU INSTITUTE OF TECHNOLOGY

COURSE: PROGRAMMING IN C

TRAINER NAME: PUGAZH MARA

SUBMISSION DATE: 25.2.2026

ABSTRACT:

This project presents a Password Utility System developed using the C programming language to enhance user security by evaluating and generating strong passwords. It addresses the common issue of weak and easily guessable passwords, which can lead to security vulnerabilities and unauthorized access. The system provides two main functionalities: password strength checking and password generation. The strength checker evaluates passwords based on length, inclusion of numeric digits, and special characters, and classifies them as weak, medium, or strong to help users improve their password quality.

In addition, the system includes a password generator with two modes: a fully random generator and a customized generator based on a user-provided keyword. The random generator creates secure passwords using a mix of letters, numbers, and special symbols, while the custom generator combines user input with random elements to produce personalized yet secure passwords. Overall, this project demonstrates how basic programming concepts can be used to address real-world security concerns and promote better password practices.

INTRODUCTION:

In today's digital world, weak passwords are a major security risk that can lead to unauthorized access and data breaches. Many users create simple passwords without considering security, making their accounts vulnerable.

This project solves this problem by providing a Password Utility System that checks password strength and generates secure passwords. It helps users create strong passwords easily and improves awareness of safe password practices, making it a simple and useful security tool.

OBJECTIVES:

· To design and develop a Password Utility System using C programming.

· To understand the use of loops for iterating through password characters to check numbers and special symbols.

· To apply conditional statements (if-else, switch) for decision-making, such as classifying password strength and handling user choices.

· To implement and understand functions like hasNumber(), hasSpecialChar(), and password generators for modular and reusable code.

· To learn the use of string handling functions (like strlen) for processing passwords.

· To generate secure passwords using random number functions (rand(), srand()) and improve password security.

· To develop a simple menu-driven console application that performs multiple operations efficiently.

· To enhance programming and problem-solving skills by applying concepts to a real-world security application.

TOOLS AND TECHNOLOGY:

- **PROGRAMMING LANGUAGE**: C
- **COMPILER**: TURBO C,VS CODE,ONLINE COMPILER
- **PLATFORM:**WINDOWS/LINUX

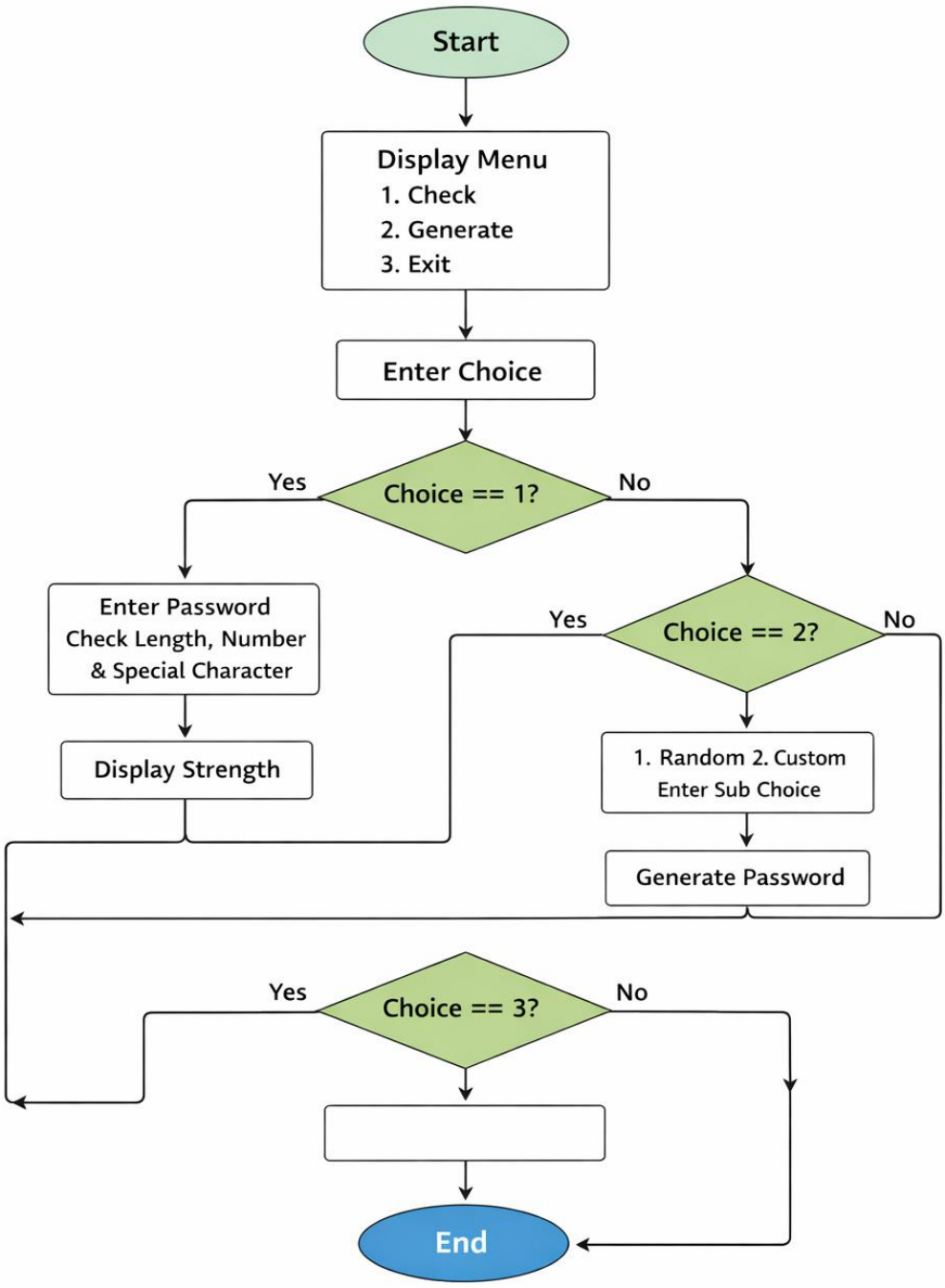
SYSTEM REQUIREMENTS:

- HARDWARE : BASIC COMPUTER WITH MINIMUM 4GB RAM
- SOFTWARE: C COMILER AND ANY IDE/TEXT EDITOR\

ALGORITHM:

- Start the program.
- Display menu with options to check password strength, generate password,or exit.
- Read the user's choice.
- If the choice is 1, read the password, calculate its length, check for numbers and special characters, display the details, and determine whether it is weak, medium, or strong based on given conditions.
- If the choice is 2, display the password generation options and read the sub-choice.
- If the sub-choice is 1, generate a random password using letters, numbers, and special characters and display it.
- If the sub-choice is 2, read a keyword, generate random numbers and a special character, combine them with the keyword, and display the custom password.
- If any invalid option is selected, display an error message.
- If the choice is 3, display the exit message and terminate the program.
- Repeat the process until the user chooses to exit.
- End the program.

FLOW CHART:


Start
Display Menu
1. Check
2. Generate
3. Exit
Enter Choice
Choice == 1?
Yes
No
Enter Password
Check Length, Number
& Special Character
Display Strength
Choice == 2?
Yes
No
1. Random 2. Custom
Enter Sub Choice
Generate Password
Choice == 3?
Yes
No
End

SOURCE CODE:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <time.h>

int hasNumber(char pwd[]) {
    for (int i = 0; pwd[i] != '\0'; i++) {
        if (pwd[i] >= '0' && pwd[i] <= '9')
            return 1;
    }
    return 0;
}

int hasSpecialChar(char pwd[]) {
    char special[] = "!@#$%^&*";
    for (int i = 0; pwd[i] != '\0'; i++) {
        for (int j = 0; special[j] != '\0'; j++) {
            if (pwd[i] == special[j])
                return 1;
        }
    }
    return 0;
}

/* Fully random password */
void generateRandomPassword() {
    char chars[] =
"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz012345678
9!@#$%^&*";
    char password[12];

    srand(time(NULL));

    for (int i = 0; i < 10; i++) {
        password[i] = chars[rand() % strlen(chars)];
    }
    password[10] = '\0';

    printf("\nGenerated Random Password: %s\n", password);
}

/* Customized password using user keyword */
void generateCustomPassword() {
    char keyword[30];
    char special[] = "!@#$%^&*";
    int num1, num2;
```

```
    srand(time(NULL));

    printf("\nEnter a keyword (name or word): ");
    scanf("%s", keyword);

    num1 = rand() % 10;
    num2 = rand() % 10;
    char sp = special[rand() % strlen(special)];

    printf("\nGenerated Custom Password: %s%c%d%d\n",
          keyword, sp, num1, num2);
}

int main() {
    char password[50];
    int length, num, special;
    int choice, subChoice;

    while (1) {
        printf("\n===== PASSWORD UTILITY =====\n");
        printf("1. Check Password Strength\n");
        printf("2. Generate Password\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {

        case 1:
            printf("\nEnter your password: ");
            scanf("%s", password);

            length = strlen(password);
            num = hasNumber(password);
            special = hasSpecialChar(password);

            printf("\nPassword Analysis:\n");
            printf("Length: %d\n", length);
            printf("Contains Number: %s\n", num ? "Yes" : "No");
            printf("Contains Special Character: %s\n", special ? "Yes" : "No");

            if (length < 6) {
                printf("Password Strength: WEAK\n");
            }
            else if (length >= 6 && num && !special) {
                printf("Password Strength: MEDIUM\n");
            }
            else if (length >= 8 && num && special) {
                printf("Password Strength: STRONG\n");
            }
```

```
            else {
                printf("Password Strength: WEAK\n");
            }
            break;

        case 2:
            printf("\nPassword Generation Mode:\n");
            printf("1. Fully Random Password\n");
            printf("2. Custom Password (Keyword Based)\n");
            printf("Enter your choice: ");
            scanf("%d", &subChoice);

            if (subChoice == 1) {
                generateRandomPassword();
            }
            else if (subChoice == 2) {
                generateCustomPassword();
            }
            else {
                printf("\nInvalid option!\n");
            }
            break;

        case 3:
            printf("\nThank you for using Password Utility!\n");
            return 0;

        default:
            printf("\nInvalid choice! Please try again.\n");
        }
    }
}
```

OUTPUT:

```
===== PASSWORD UTILITY =====
1. Check Password Strength
2. Generate Password
3. Exit
Enter your choice: 2

Password Generation Mode:
1. Fully Random Password
2. Custom Password (Keyword Based)
Enter your choice: DHARAN

Invalid option!

===== PASSWORD UTILITY =====
1. Check Password Strength
2. Generate Password
3. Exit
Enter your choice:
Password Generation Mode:
1. Fully Random Password
2. Custom Password (Keyword Based)
Enter your choice:
Invalid option!

===== PASSWORD UTILITY =====
1. Check Password Strength
2. Generate Password
3. Exit
Enter your choice:
Password Generation Mode:
1. Fully Random Password
2. Custom Password (Keyword Based)
Enter your choice:
Invalid option!
```

RESULT:

THE PROGRAM EXECUTED SUCCESSFULLY AND PRODUCED THE EXPECTED OUTPUT.

APPLICATIONS:

- Used to check password strength to prevent weak and easily guessable passwords.
- Helps users to create secure passwords using random and customized generation methods.
- Can be used in login systems to enforce strong password policies.
- Useful in educational purposes to understand security concepts in programming.
- Can be applied in websites and applications for improving user account security.
- Helps in cybersecurity awareness by guiding users to use strong passwords.
- Can be integrated into software systems where user authentication is required

CONCLUSION:

This project demonstrates a simple and effective way to evaluate and generate secure passwords using basic C programming concepts. It helps improve user awareness about password security and promotes the use of strong passwords to prevent unauthorized access.

FUTURE ENHANCEMENTS:

- Add a graphical user interface (GUI) to make the system more user-friendly.
- Store passwords securely using file handling or encryption for real-world usage.
- Include more advanced checks like uppercase, lowercase, and password patterns.
- Allow users to save and manage multiple passwords like a basic password manager.

REFERENCE:

Balagurusamy, E., *Programming in ANSI C*, McGraw Hill Edu